



UNIVERSIDAD DON BOSCO

Facultad de Ingeniería – Técnico en Ingeniería en Ciencias de la Computación

Integrantes:

Katherine Alexandra Pinto Vila	PV251591
Grecia Alejandra Soto Parada	SP250245
Omar Gabriel Martínez Murcia	MM252309

Docente:

Ing. Rafael Alexander Torres Rodríguez

Actividad:

Sistema de Gestión de Inventario Web

(Proyecto Java Web MVC con JSP, Servlets, DAO, JDBC, JSTL y MySQL)

Asignatura:

programación Orientada a Objetos - POO404

Fecha:

2026

INDICE

INDICE.....	2
1. INTRODUCCION.....	3
Objetivo general.....	3
Objetivos especificos.....	3
2. MARCO TEORICO	4
JSTL	4
MVC.....	4
DAO	4
JDBC	4
3. Análisis del sistema desarrollado.....	5
Alcance funcional	5
4. Diagrama E-R de la base de datos.....	6
5. Diagrama de arquitectura del proyecto MVC	7
6. Descripción de cada capa	9
Capa de modelo	9
Capa de conexión.....	9
Capa DAO	10
Capa de controlador.....	10
Capa de vista	11
Validación cliente y servidor	11
7. Evidencia	11
<i>categories</i>	13
<i>productos</i>	14
<i>formulario de producto</i>	15
8. Conclusión	16
Recomendaciones	17
9. Bibliografía	18

1. INTRODUCCION

El sistema de gestión de Inventario Web es una aplicación Java orientada a administrar categorías y productos mediante operaciones CRUD. El proyecto está organizado como una aplicación web Maven empaquetada en WAR, preparada para ejecutarse sobre Apache Tomcat 10.x y conectarse a una base de datos MySQL llamada db_inventario.

La solución aplica una arquitectura MVC sencilla: las páginas JSP representan la vista, los Servlets atienden las peticiones HTTP y coordinan el flujo, las clases modelo encapsulan los datos del dominio, y los DAO concentran el acceso persistente mediante JDBC. Esta separación permite que el sistema sea comprensible, mantenible y extensible para nuevos módulos.

Objetivo general

Documentar técnica y funcionalmente el proyecto inventarioweb, explicando su estructura, tecnologías utilizadas, diseño de base de datos, arquitectura MVC, funcionamiento por capas y evidencias visuales de las funcionalidades implementadas.

Objetivos específicos

- ✓ Describir las tecnologías usadas en el proyecto: Jakarta Servlet, JSP, JSTL, JDBC, MySQL y Maven.
- ✓ Explicar el modelo E-R que soporta la gestión de categorías y productos.
- ✓ Relacionar cada capa del patrón MVC con los archivos reales del proyecto.
- ✓ Presentar fragmentos de Código relevantes que demuestran validación, persistencia, navegación y vistas dinámicas.
- ✓ Evidenciar las funcionalidades principales: inicio, listados, formularios, filtros, edición y eliminación lógica.

2. MARCO TEORICO

JSTL

JSTL, actualmente Jakarta Standard Tag Library en el ecosistema Jakarta EE, proporciona etiquetas reutilizables para JSP. En este proyecto se observan etiquetas del núcleo y formato, por ejemplo, `c:forEach`, `c:if`, `c:choose`, `c:url`, `c:out` y `fmt:formatDate`. Su uso reduce la mezcla de código Java dentro de las páginas y favorece vistas más limpias.

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>

<c:forEach items="${lista}" var="p" varStatus="s">
    <td><c:out value="${p.nombre}" /></td>
    <td><fmt:formatNumber value="${p.precio}" minFractionDigits="2"
/></td>
</c:forEach>
```

MVC

El patrón Modelo-Vista-Controlador separa la interfaz, el control de flujo y los datos. En aplicaciones web con Servlets y JSP, el controlador suele recibir la solicitud, consultar o modificar el modelo y reenviar a una vista JSP. En `inventarioweb`, `CategoriaServlet` y `ProductoServlet` reciben acciones como listar, nuevo, editar, eliminar, guardar y actualizar.

DAO

El patrón Data Access Object abstrae y encapsula el acceso a la fuente de datos. La ventaja central es que el resto de la aplicación no necesita conocer detalles de SQL, conexiones o `ResultSet`. En el proyecto, `CategoriaDAO` y `ProductoDAO` contienen los métodos listar, insertar, actualizar, eliminar y validar duplicados.

JDBC

JDBC es la API de Java para conectarse a bases de datos relacionales, ejecutar consultas SQL y procesar resultados. El proyecto usa `DriverManager` para abrir conexiones MySQL y `PreparedStatement` para enviar parámetros, lo que mejora la legibilidad y reduce riesgos de inyección SQL frente a concatenar cadenas.

3. Análisis del sistema desarrollado

El proyecto esta ubicado bajo el paquete edu.udb.inventarioweb y utiliza una estructura Maven convencional. Los artefactos fuente principales se ubican en src/main/java, las vistas en src/main/webapp y el script de base de datos en src/main/resources.

Elemento	Archivo o carpeta	Funcion
Modelo	modelo/Categoria.java y modelo/Producto.java	Representan las entidades del dominio y sus atributos.
DAO	dao/CategoriaDAO.java, dao/ProductoDAO.java, dao/ConexionDB.java	Aislan la conexión y las operaciones SQL.
Controladores	controlador/CategoriaServlet.java y controlador/ProductoServlet.java	Reciben acciones HTTP y coordinan vistas y DAOs.
Vistas	index.jsp y vistas/*//*.jsp	Presentan formularios, listados, alertas y datos formateados.
Recursos	css/estilos.css y js/validaciones.js	Definen la apariencia y validaciones del lado cliente.
Base de datos	db_inventario.sql	Crea tablas, restricciones e información inicial.

Alcance funcional

- ✓ Gestión de categorías: listado, registro, edición, cambio de estado y eliminación logica.
- ✓ Gestión de productos: listado, registro, edición, cambio de estado, eliminación logica y filtro por categoría.
- ✓ Validaciones de negocio: nombre de categoría unico, codigo de producto único, precio mayor a cero y stock no negativo.
- ✓ Validaciones de integridad: una categoría con productos activos no puede eliminarse desde la interfaz del sistema.
- ✓ Formato visual: etiquetas de estado, alerta de mensajes, formato de fecha y precio, y resaltado de stock bajo.

4. Diagrama E-R de la base de datos

Enlace; https://lucid.app/lucidchart/a03b56cb-b78f-4808-aeafa9db1c9aada6/edit?viewport_loc=-2297%2C333%2C4731%2C2145%2C0_0&invitationId=inv_cd072ba1-9435-467a-824896cea6947fbb

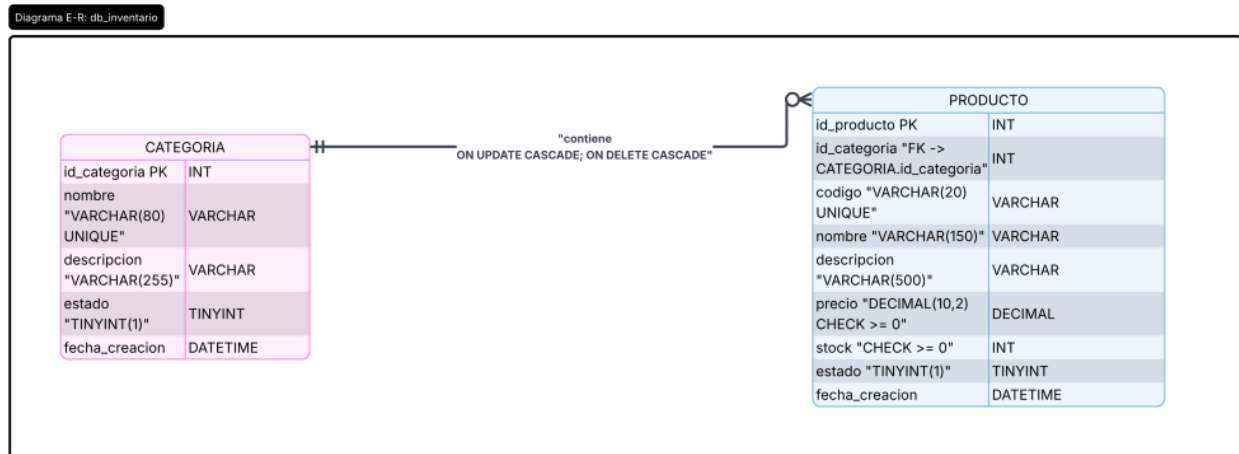


Figura 1. Modelo entidad-relación

La base de datos tiene dos entidades principales. CATEGORIA almacena clasificaciones de productos y PRODUCTO almacena los artículos del inventario. La relación es de uno a muchos: una categoría puede asociarse con varios productos, mientras que cada producto pertenece a una categoría obligatoria.

Tabla	Clave primaria	Restricciones principales
CATEGORIA	id_categoria	nombre único, estado por defecto 1, fecha_creacion automática.
PRODUCTO	id_producto	código unico, FK id_categoria, precio >= 0, stock >= 0, eliminación restringida por relacion.

5. Diagrama de arquitectura del proyecto MVC

Enlace; https://lucid.app/lucidchart/a03b56cb-b78f-4808-aeaf/a9db1c9aada6/edit?view_items=xD60pQqWf0~e%2CD6002lmYkSW%2CD60U4xiQXnw%2CD6083_AC2a.%2CD60UmwxfN~%2CD60TkIn2FSK%2CD60IEomIxmy%2CD60grTlZDdj%2CD60mN5s9IEF%2CD60iU0bhTfE%2CD60rVjDcpuq%2CD60Bsio0kR5%2CD60paT~EwCZ%2CD60zgjAIVMy%2CD60.zjKxcZh%2CD60iA_kd7po%2CD60OiuRisTE&page=jC601~flKtx-&invitationId=inv_cd072ba1-9435-467a-8248-96eea6947fbb

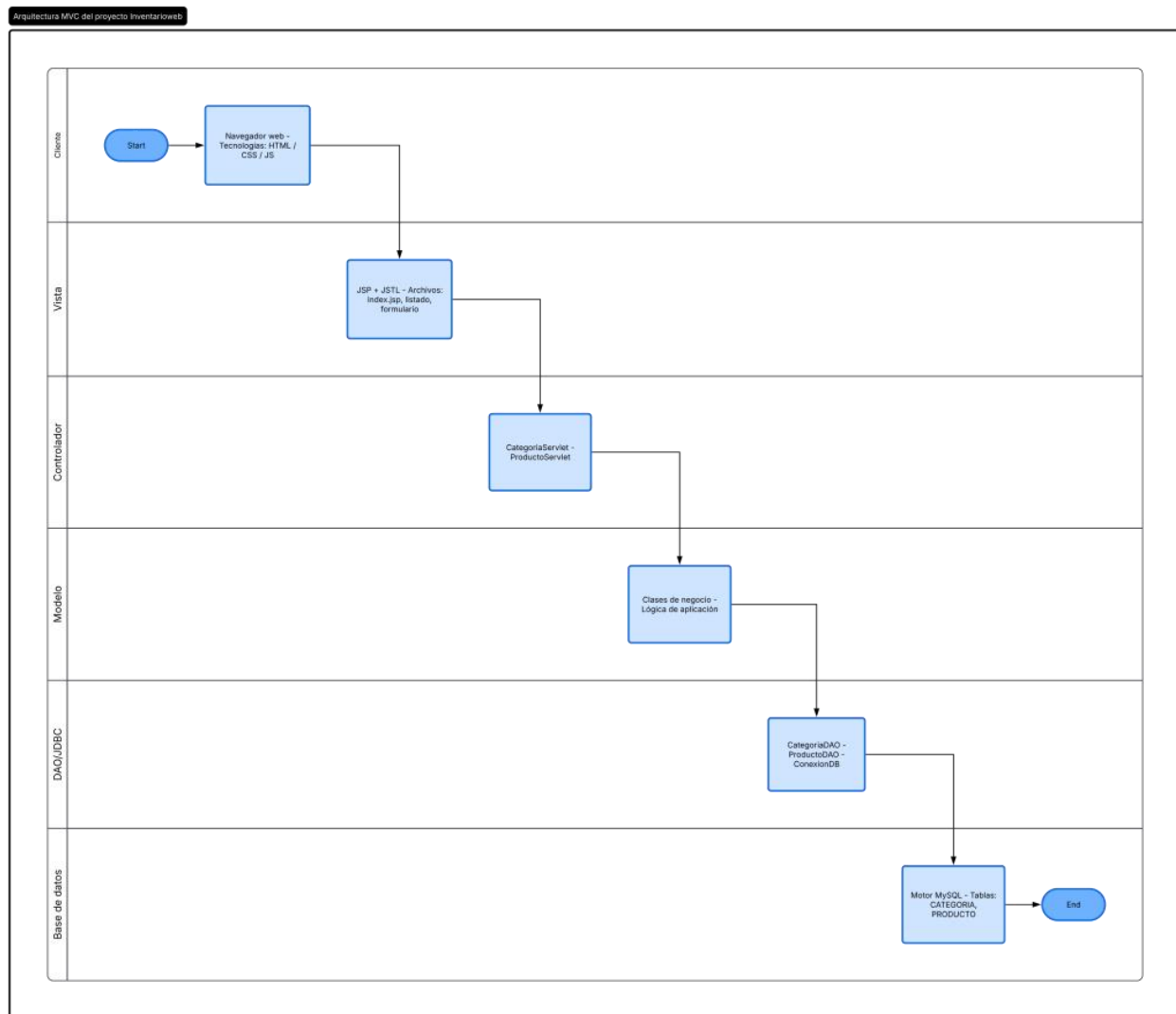


Figura 2. Vista general de la arquitectura MVC implementada.

La arquitectura inicia en el navegador, donde el usuario interactúa con enlaces y formularios. Las solicitudes llegan a los Servlets mediante rutas anotadas con WebServlet. El controlador interpreta la acción, invoca DAO cuando necesita datos y finalmente redirige o reenvía hacia JSP. Las JSP consumen atributos de request y sesión para renderizar contenido dinámico con JSTL.

Enlace; https://lucid.app/lucidchart/a03b56cb-b78f-4808-acafa9db1c9aada6/edit?viewport_loc=-2104%2C-933%2C5266%2C2388%2CqI609wey9jkE&invitationId=inv_cd072ba1-9435-467a-8248-96eea6947fbb

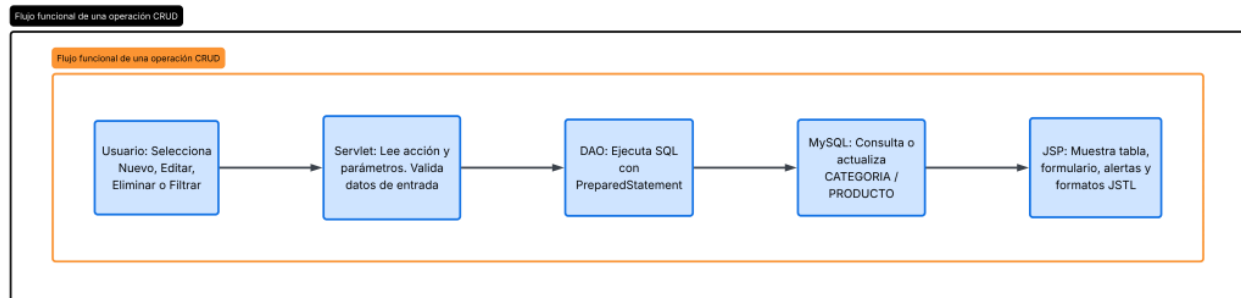


Figura 3. Flujo de una operación CRUD típica dentro de inventarioweb.

6. Descripción de cada capa

Capa de modelo

Los modelos son clases Java simples que representan la información persistida y transportada entre capas. categoría contiene idCategoría, nombre, descripción, estado y fechaCreacion. Producto agrega idProducto, idCategoría, código, precio, stock y nombreCategoría, atributo útil para mostrar el nombre de la categoría en el listado.

```
public class Producto {  
    private int idProducto;  
    private int idCategoría;  
    private String código;  
    private String nombre;  
    private double precio;  
    private int stock;  
    private String nombreCategoría;  
}
```

Capa de conexión

ConexionDB centraliza los datos de conexión JDBC. El proyecto carga el driver com.mysql.cj.jdbc.Driver y usa DriverManager para conectarse a db_inventario en localhost. Aunque esta solución es adecuada para un laboratorio, en producción se recomienda mover credenciales a variables de entorno o un pool de conexiones.

```
Class.forName("com.mysql.cj.jdbc.Driver");  
Connection con = DriverManager.getConnection(URL, USER, PASSWORD);
```

Capa DAO

Los DAO contienen la lógica SQL y usan try-with-resources para cerrar Connection, PreparedStatement y ResultSet. En ProductoDAO, el listado usa INNER JOIN para traer también el nombre de la categoría; de esa forma la vista no necesita realizar consultas adicionales.

```
String sql = "SELECT p.*, c.nombre AS categoria FROM PRODUCTO p " +
            "INNER JOIN CATEGORIA c ON p.id_categoria = c.id_categoria "
+
            "ORDER BY p.nombre";

try (Connection con = ConexionDB.getConexion();
    PreparedStatement ps = con.prepareStatement(sql);
    ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        Producto p = new Producto();
        p.setCodigo(rs.getString("codigo"));
        p.setNombreCategoria(rs.getString("categoria"));
        lista.add(p);
    }
}
```

Capa de controlador

Los controladores gestionan acciones. En ProductoServlet se identifica el parámetro accion y se decide si se mostrara el formulario, se cargará un registro existente, se eliminará lógicamente o se listarán productos. La validación del lado servidor evita guardar datos incompletos o incoherentes.

```
String accion = request.getParameter("accion");
if (accion == null) accion = "listar";

switch (accion) {
    case "nuevo":
        request.setAttribute("categorias",
            categoriaDAO.listarActivas());
        request.getRequestDispatcher("vistas/producto/formulario.jsp")
            .forward(request, response);
        break;
    case "eliminar":
        productoDAO.eliminar(idEliminar);
        response.sendRedirect("producto?accion=listar");
        break;
}
```

Capa de vista

Las vistas JSP se apoyan en JSTL para recorrer listas, escapar contenido y construir URLs con el contexto de la aplicación. Esto evita rutas rígidas y reduce la probabilidad de errores al desplegar el WAR con un nombre de contexto específico.

```
<c:url var="urlEditar" value="/producto">
    <c:param name="accion" value="editar" />
    <c:param name="id" value="${p.idProducto}" />
</c:url>

<a href="${urlEditar}" class="btn-custom btn-warning-custom">Editar</a>
```

Validación cliente y servidor

El archivo validaciones.js realiza una primera revisión en el navegador. Sin embargo, las validaciones decisivas se mantienen en el servidor, dentro de los Servlets, porque el usuario podría desactivar JavaScript o enviar peticiones manuales.

```
if (p.getPrecio() <= 0) {
    errores.put("precio", "El precio debe ser mayor a 0.");
}

if (p.getStock() < 0) {
    errores.put("stock", "El stock debe ser mayor o igual a 0.");
}
```

7. Evidencia

Las capturas se generaron como evidencias visuales basadas en las JSP, estilos CSS y datos iniciales del script SQL del proyecto. Representan el comportamiento esperado al desplegar la aplicación en Tomcat con la base de datos cargada.

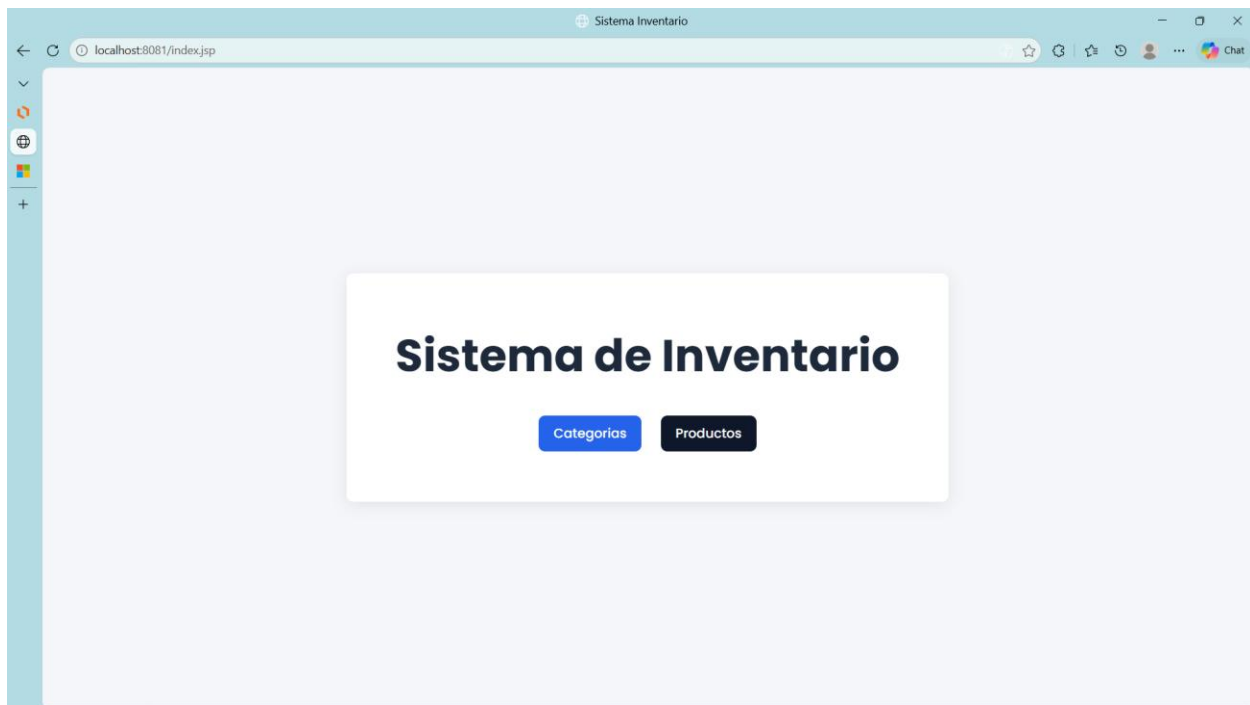


Figura 4. Pantalla de inicio con acceso a categorías y productos.

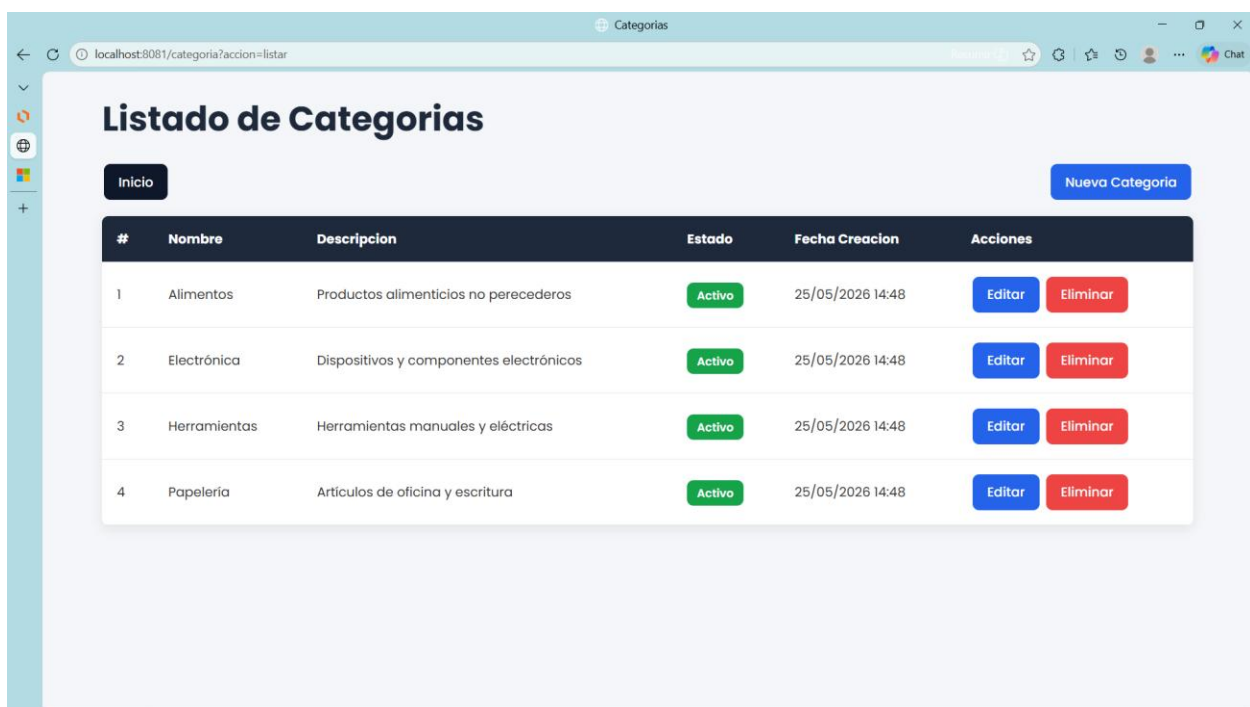


Figura 5. Listado de categorías con acciones de edición y eliminación.

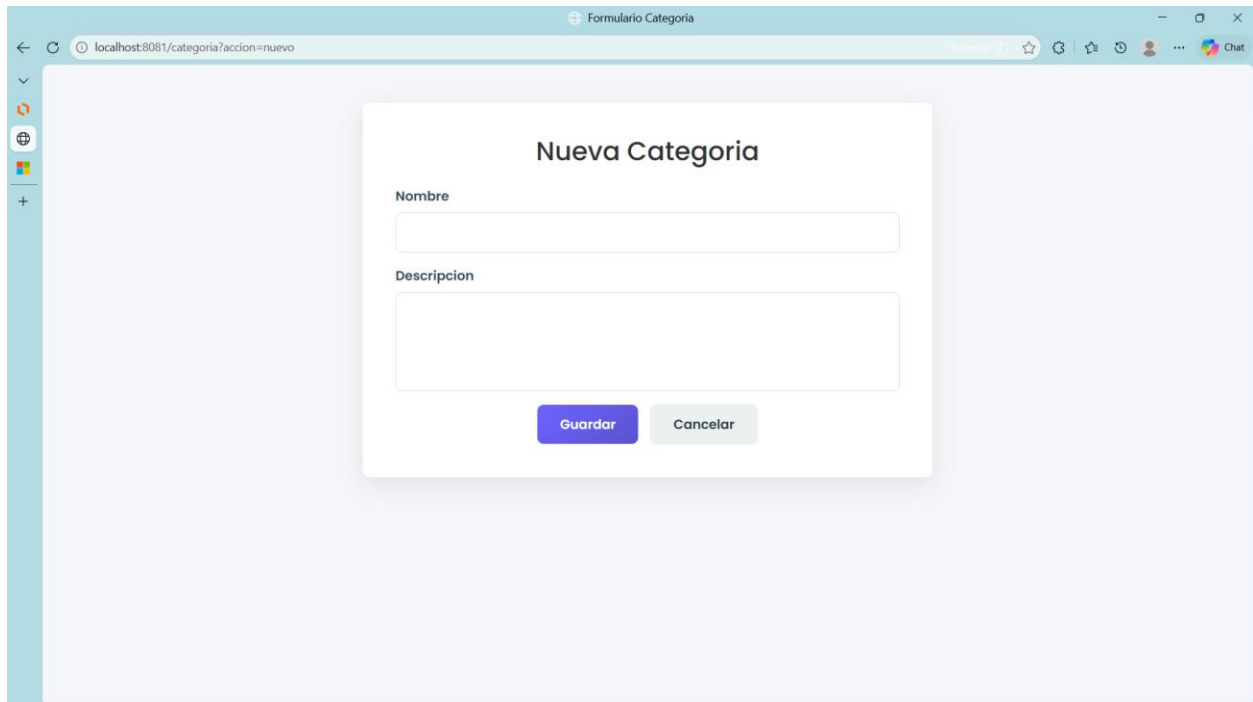
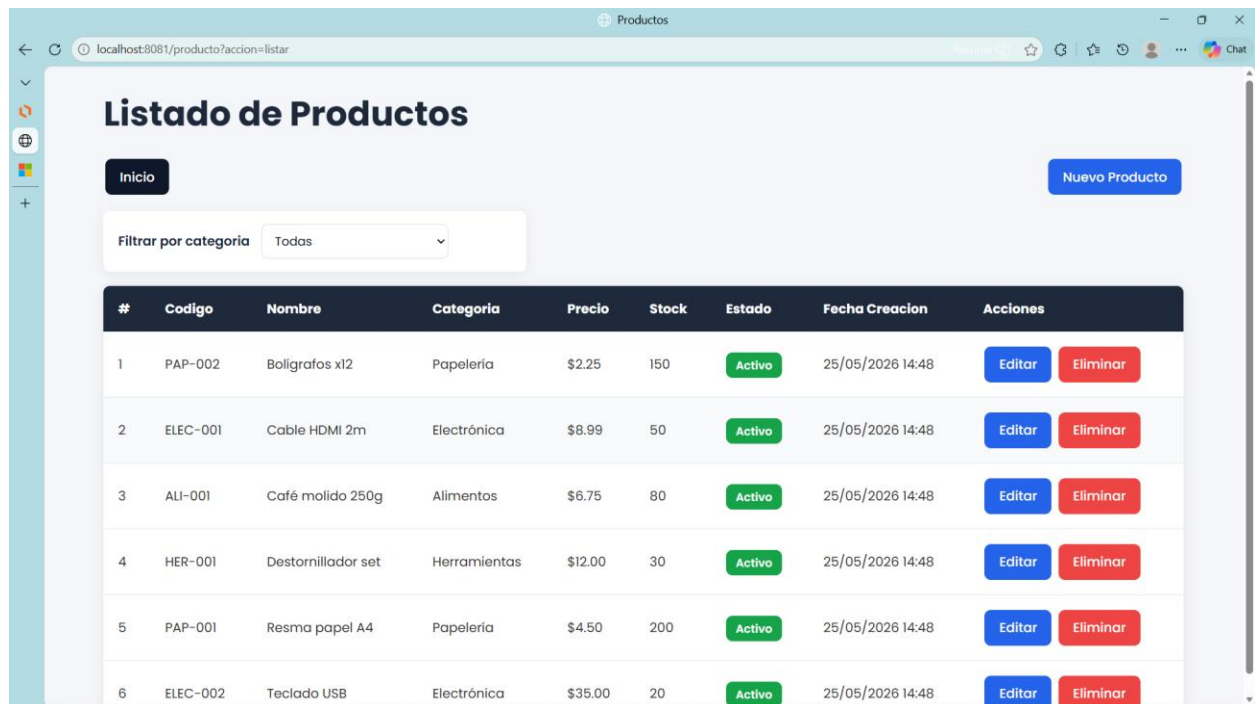


Figura 6. Formulario de categoría para crear o editar registros.

La gestión de categorías contempla nombre obligatorio, longitud máxima y verificación de nombre duplicado en el servidor. Cuando el usuario intenta eliminar una categoría con productos activos, CategoríaServlet consulta tieneProductosActivos y presenta un mensaje de error.

```
if (dao.tieneProductosActivos(idEliminar)) {  
    request.getSession().setAttribute("error",  
        "No se puede eliminar la categoria porque tiene productos  
activos.");  
} else if (dao.eliminar(idEliminar)) {  
    request.getSession().setAttribute("mensaje",  
        "Categoria eliminada correctamente.");  
}
```

productos



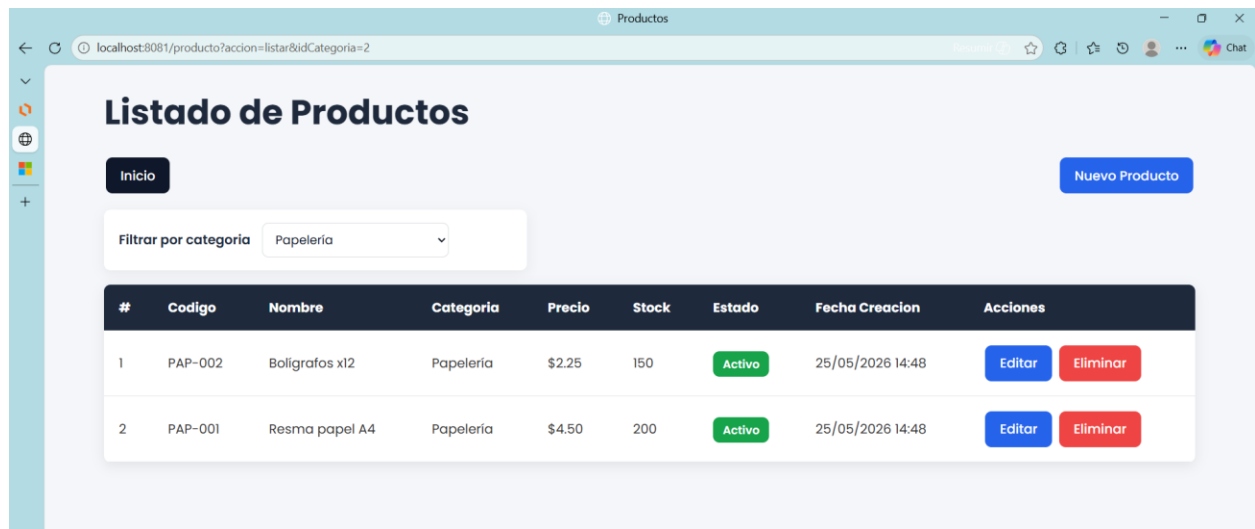
Listado de Productos

Inicio Nuevo Producto

Filtrar por categoría: Todos

#	Código	Nombre	Categoría	Precio	Stock	Estado	Fecha Creacion	Acciones
1	PAP-002	Bolígrafos x12	Papelería	\$2.25	150	Activo	25/05/2026 14:48	Editar Eliminar
2	ELEC-001	Cable HDMI 2m	Electrónica	\$8.99	50	Activo	25/05/2026 14:48	Editar Eliminar
3	ALI-001	Café molido 250g	Alimentos	\$6.75	80	Activo	25/05/2026 14:48	Editar Eliminar
4	HER-001	Destornillador set	Herramientas	\$12.00	30	Activo	25/05/2026 14:48	Editar Eliminar
5	PAP-001	Resma papel A4	Papelería	\$4.50	200	Activo	25/05/2026 14:48	Editar Eliminar
6	ELEC-002	Teclado USB	Electrónica	\$35.00	20	Activo	25/05/2026 14:48	Editar Eliminar

Figura 7. Listado general de productos con precio, stock, categoría y estado.



Listado de Productos

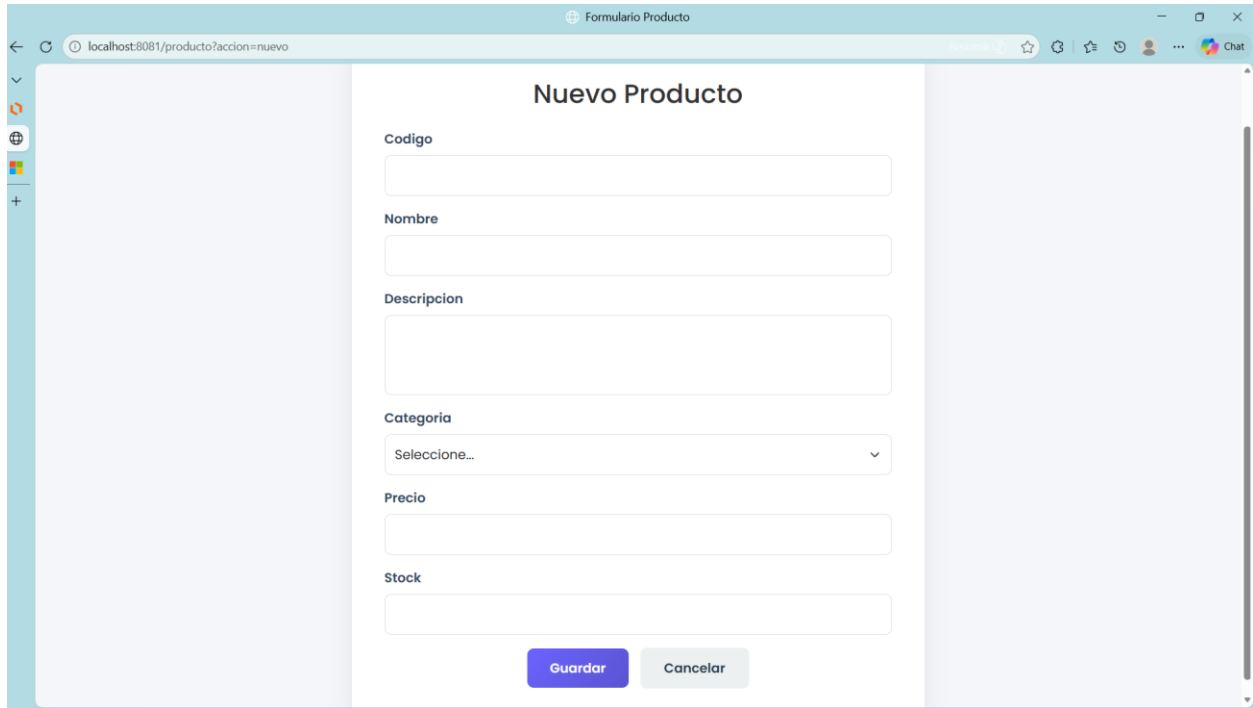
Inicio Nuevo Producto

Filtrar por categoría: Papelería

#	Código	Nombre	Categoría	Precio	Stock	Estado	Fecha Creacion	Acciones
1	PAP-002	Bolígrafos x12	Papelería	\$2.25	150	Activo	25/05/2026 14:48	Editar Eliminar
2	PAP-001	Resma papel A4	Papelería	\$4.50	200	Activo	25/05/2026 14:48	Editar Eliminar

Figura 8. Filtro de productos por categoría.

formulario de producto



The screenshot shows a web browser window with the title 'Formulario Producto'. The address bar displays 'localhost:8081/producto?accion=nuevo'. The main content area is titled 'Nuevo Producto' and contains a form with the following fields: 'Codigo' (text input), 'Nombre' (text input), 'Descripcion' (text area), 'Categoria' (dropdown menu with 'Seleccione...' as the selected option), 'Precio' (text input), and 'Stock' (text input). At the bottom of the form are two buttons: 'Guardar' (blue) and 'Cancelar' (gray). The browser's sidebar on the left shows various icons, and the top right corner has a 'Chat' icon.

Figura 9. Formulario para registrar o editar productos.

El formulario de producto solicita código, nombre, descripción, categoría, precio y stock. En modo edición se muestra también el campo estado, lo que permite reactivar o inactivar un registro. El código se normaliza a mayúsculas antes de validar y guardar.

```
p.setCodigo(limpiar(request.getParameter("codigo")).toUpperCase());

if (!p.getCodigo().matches("[A-Z0-9-]{3,20}")) {
    errores.put("codigo",
        "El codigo debe tener de 3 a 20 caracteres alfanumericos o guiones.");
}
```

8. Conclusión

El proyecto logra establecer de manera efectiva una separación por capas bajo el patrón MVC, lo que permite definir responsabilidades claras y facilita el mantenimiento y la evolución del código. La incorporación de JSTL en las vistas JSP aporta mayor legibilidad y expresividad en la presentación de datos dinámicos, eliminando la necesidad de utilizar scriptlets y promoviendo buenas prácticas de desarrollo.

La capa DAO centraliza el acceso a la base de datos MySQL mediante JDBC y PreparedStatement, reduciendo el acoplamiento entre los controladores y las consultas SQL, y garantizando mayor seguridad frente a inyecciones. El diseño de la base de datos resulta consistente para un sistema de inventario inicial: las entidades cuentan con claves primarias, restricciones únicas y relaciones foráneas que aseguran integridad referencial. La estrategia de eliminación lógica mediante un campo de estado permite conservar el historial de registros y evita la pérdida de información relevante.

Asimismo, las validaciones en cliente y servidor fortalecen la coherencia de los datos y mejoran la experiencia del usuario, asegurando que las operaciones CRUD se ejecuten de manera confiable. En conjunto, estas decisiones de diseño y desarrollo reflejan un sistema robusto, escalable y defendible en un entorno académico, con potencial de adaptación a escenarios reales de administración.

Recomendaciones

Gestión de credenciales

- ✓ Mover las credenciales de ConexionDB a **variables de entorno** o a un archivo de configuración externo no versionado (ej. .env), evitando exponer datos sensibles en el repositorio. (Esto fortalece la seguridad y facilita la portabilidad del sistema)

Optimización de conexiones

- ✓ Implementar un **pool de conexiones** mediante DataSource del contenedor o librerías como HikariCP. (Esta práctica mejora el rendimiento y la escalabilidad cuando aumente el número de usuarios concurrentes)

Pruebas y calidad de software

- ✓ Incluir **pruebas unitarias** para validar la lógica de negocio y las reglas de entrada.
- ✓ Incorporar **pruebas de integración** para los DAOs utilizando una base de datos de prueba, garantizando la correcta interacción con MySQL.

Usabilidad en listados

- ✓ Agregar **paginación y búsqueda por texto** en los listados de productos y categorías. (Esto asegura un mejor rendimiento y experiencia de usuario cuando el inventario tenga miles de registros)

Gestión de errores y monitoreo

- ✓ Sustituir la impresión de *stack traces* por una **librería de logging** (ej. SLF4J + Logback).
- ✓ Configurar niveles de log (INFO, WARN, ERROR) y almacenamiento centralizado para facilitar el diagnóstico.

Seguridad y control de acceso

- ✓ Implementar **roles de usuario** (ej. administrador, vendedor, auditor) con control de permisos en cada módulo. (Esto es esencial si el sistema se usará en un entorno real de administración)

9. Bibliografía

Hartman, J. (2024, 8 noviembre). *JSTL (Biblioteca de etiquetas estándar JSP)*. Guru99.

<https://www.guru99.com/es/jsp-tag-library.html>

Informix Servers. (s. f.). [https://www.ibm.com/docs/es/informix-servers/12.10.0?topic=started-](https://www.ibm.com/docs/es/informix-servers/12.10.0?topic=started-what-is-jdbc)

[what-is-jdbc](https://www.ibm.com/docs/es/informix-servers/12.10.0?topic=started-what-is-jdbc)

Juliensuze. (2026, 25 febrero). *Data Access Object (DAO): ¿Qué es? ¿Para qué sirve?* Liora.

<https://liora.io/es/que-es>

King, P. M. (2013, 22 julio). *Introduction – Apache Maven WAR Plugin*.

<https://maven.apache.org/plugins/maven-war-plugin/>

Lucid visual collaboration suite: Log in. (s. f.).

https://lucid.app/documents#/documents?folder_id=home

Oracle JDBC frequently asked questions. (s. f.).

<https://www.oracle.com/database/technologies/faq-jdbc.html>

org.glassfish.web » jakarta.servlet.jsp.jstl » 3.0.0 - jar download | JarCasting. (s. f.).

<https://jarcasting.com/artifacts/org.glassfish.web/jakarta.servlet.jsp.jstl/3.0.0/>

Qué es MVC. (s. f.). DesarrolloWeb.com. <https://desarrolloweb.com/articulos/que-es-mvc.html>

Soomro, S. (2023, 12 octubre). *Implementación del objeto de acceso a datos en Java*. Delft

Stack. <https://www.delftstack.com/es/howto/java/dao-in-java/>